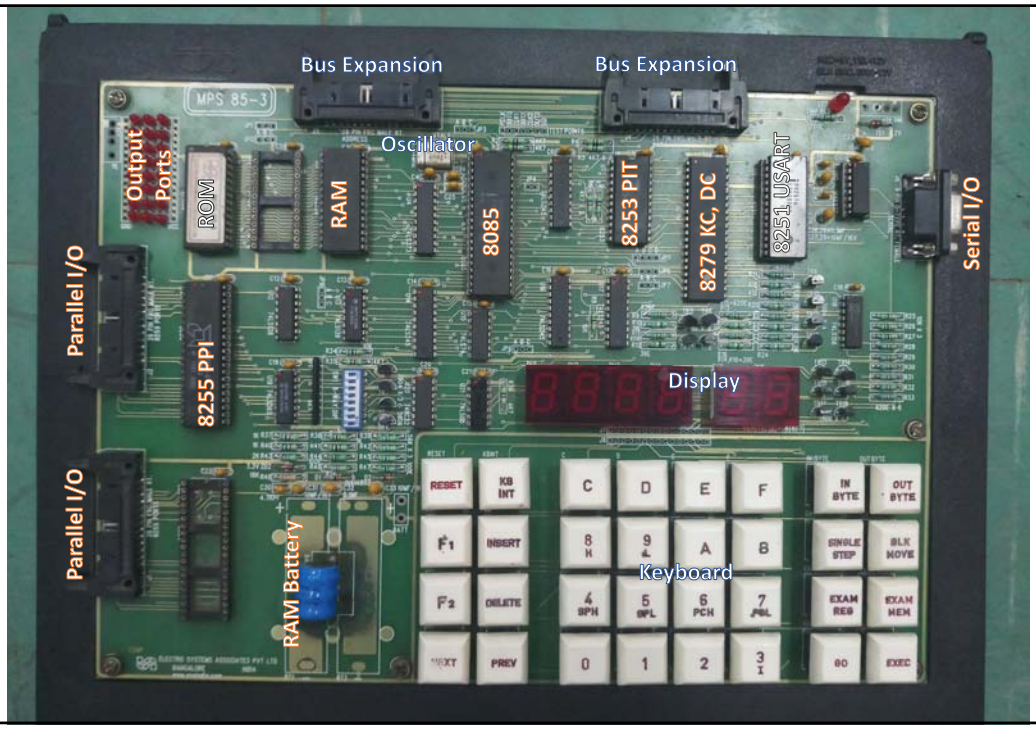
Introduction to Microprocessor Lab

DS Baral



8085 Trainer Kit

Components of 8085 µP Trainer Kit

- **CPU :**
  - 8085 operated at 3.072 MHz
- **MEMORY *:***
  - Three 28 pin JEDEC sockets offer 64K Bytes of memory
- **ROM :**
  - 16K Bytes of monitor program in 27128 EPROM
- **ROM / RAM :**
  - 16K Bytes user expansion using any EPROM/SRAM
- **RAM :**
  - 32K Bytes supplied using 62256 CMOS static RAM with battery backup

Components of 8085 µP Trainer Kit (Contd…)

- ***8255 :***
  - (Two Nos) One 8255 is supplied to give 24 programmable I/O lines.
  - User can have 48 I/O lines by populating an additional 8255 in expansion socket.
- ***8253 :***
  - 3 programmable interval timers
  - Timer 0 is used for implementing single step facility,
  - Timer 1 is used for generating baud clock
  - Timer 2 is available to the user.
- ***8251***
  - For serial communication.
  - Supports all standard bauds from 110 to 19200.

Components of 8085 µP Trainer Kit (Contd…)

- ***8279 :***
  - To Control 32 keys keyboard and 6-digit, half-inch size, seven segment LED display.
- ***Bus Expansion :***
  - Fully de-multiplexed and buffered TTL compatible bus signals brought out through two 26 pin ribbon cable connectors for expansion.
- ***INTERFACE SIGNALS :***
  - ***Parellel I/O:*** 48 TTL compatible lines (2x8255) brought out through two 26 pin ribbon cable connectors.
  - ***Serial I/O :*** RS-232-C serial interface signals through onboard 9 pin D type female connector.

Program Types

- Machine Language Program
  - Instructions are given as machine instructions (binary instructions as interpreted by the computer by its hardware)
  - Programs are written with machine instructions (binary coded instructions)
- Assembly Language Program
  - The instructions are represented by Mnemonics (short form of word for an operation such as ADD for addition)
  - Programs are written with mnemonics.
- Low Level Language Program
  - Each statement of a program is converted to a single machine instruction
- High Level Language Program
  - Each statement of a program is converted to more than one machine instruction
  - Easy to write as it uses words from human language for keywords

Instruction Set

- Each microprocessor is equipped with components that can interpret some instruction provided to it
- A set of instructions that a microprocessor can interpret is called an instruction set
- Instruction set is provided with mnemonic of a instruction and its machine code in binary (normally in hexadecimal notation for shorter representation)
- 8085 has 74 instructions (with 246 binary values)
- Each instruction is represented by 8-bit binary value
- Instruction has operation code (or opcode) and operand.

8085 Programming Architecture

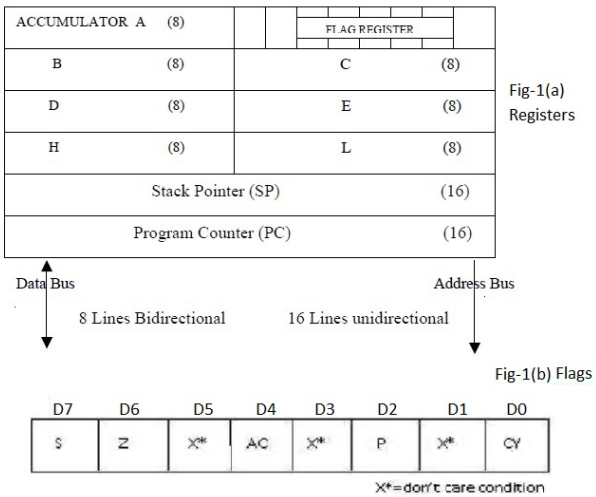
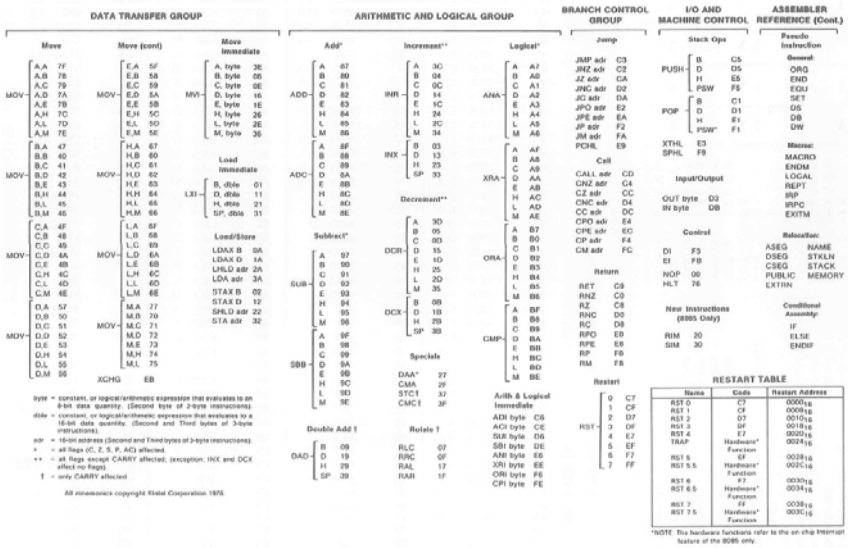


Fig-1(a) Registers

Fig-1(b) Flags

8085 Microprocessor Instruction Set



DATA TRANSFER GROUP

Move		Move (cont)		Move Immediate	
MOV A,A	7F	MOV E,A	5F	MVI A, byte	3E
MOV A,B	78	MOV E,B	58	MVI B, byte	06
MOV A,C	79	MOV E,C	59	MVI C, byte	0E
MOV A,D	7A	MOV E,D	5A	MVI D, byte	16
MOV A,E	7B	MOV E,E	5B	MVI E, byte	1E
MOV A,H	7C	MOV E,H	5C	MVI H, byte	26
MOV A,L	7D	MOV E,L	5D	MVI L, byte	2E
MOV A,M	7E	MOV E,M	5E	MVI M, byte	36
MOV B,A	47	MOV H,A	67		
MOV B,B	40	MOV H,B	60		
MOV B,C	41	MOV H,C	61		
MOV B,D	42	MOV H,D	62		
MOV B,E	43	MOV H,E	63		
MOV B,H	44	MOV H,H	64		
MOV B,L	45	MOV H,L	65		
MOV B,M	46	MOV H,M	66		
MOV C,A	4F	MOV L,A	6F		
MOV C,B	48	MOV L,B	68		
MOV C,C	49	MOV L,C	69		
MOV C,D	4A	MOV L,D	6A		
MOV C,E	4B	MOV L,E	6B		
MOV C,H	4C	MOV L,H	6C		
MOV C,L	4D	MOV L,L	6D		
MOV C,M	4E	MOV L,M	6E		
MOV D,A	57	MOV M,A	77		
MOV D,B	50	MOV M,B	70		
MOV D,C	51	MOV M,C	71		
MOV D,D	52	MOV M,D	72		
MOV D,E	53	MOV M,E	73		
MOV D,H	54	MOV M,H	74		
MOV D,L	55	MOV M,L	75		
MOV D,M	56	XCHG	EB		

Load Immediate

LXI B, dble	01
LXI D, dble	11
LXI H, dble	21
LXI SP, dble	31

Load/Store

LDAX B	0A
LDAX D	1A
LHLD adr	2A
LDA adr	3A
STAX B	02
STAX D	12
SHLD adr	22
STA adr	32

byte = constant, or logical/arithmetic expression that evaluates to an 8-bit data quantity. (Second byte of 2-byte instructions).
dble = constant, or logical/arithmetic expression that evaluates to a 16-bit data quantity. (Second and Third bytes of 3-byte instructions).
adr = 16-bit address (Second and Third bytes of 3-byte instructions).
* = all flags (C, Z, S, P, AC) affected.
** = all flags except CARRY affected; (exception: INX and DCX affect no flags).
† = only CARRY affected.

All mnemonics copyright Intel Corporation 1976.

ARITHMETIC AND LOGICAL GROUP

Add*		Increment**		Logical*	
ADD A	87	INR A	3C	ANA A	A7
ADD B	80	INR B	04	ANA B	A0
ADD C	81	INR C	0C	ANA C	A1
ADD D	82	INR D	14	ANA D	A2
ADD E	83	INR E	1C	ANA E	A3
ADD H	84	INR H	24	ANA H	A4
ADD L	85	INR L	2C	ANA L	A5
ADD M	86	INR M	34	ANA M	A6
ADC A	8F	INX B	03	XRA A	AF
ADC B	88	INX D	13	XRA B	A8
ADC C	89	INX H	23	XRA C	A9
ADC D	8A	INX SP	33	XRA D	AA
ADC E	8B			XRA E	AB
ADC H	8C			XRA H	AC
ADC L	8D			XRA L	AD
ADC M	8E			XRA M	AE
Subtract*		**Decrement****		ORA A	B7
SUB A	97	DCR A	3D	ORA B	B0
SUB B	90	DCR B	05	ORA C	B1
SUB C	91	DCR C	0D	ORA D	B2
SUB D	92	DCR D	15	ORA E	B3
SUB E	93	DCR E	1D	ORA H	B4
SUB H	94	DCR H	25	ORA L	B5
SUB L	95	DCR L	2D	ORA M	B6
SUB M	96	DCR M	35	CMP A	BF
SBB A	9F	DCX B	0B	CMP B	B8
SBB B	98	DCX D	1B	CMP C	B9
SBB C	99	DCX H	2B	CMP D	BA
SBB D	9A	DCX SP	3B	CMP E	BB
SBB E	9B			CMP H	BC
SBB H	9C			CMP L	BD
SBB L	9D			CMP M	BE
SBB M	9E				

Double Add †

DAD B	09
DAD D	19
DAD H	29
DAD SP	39

Specials

DAA*	27
CMA	2F
STC†	37
CMC†	3F

Rotate †

RLC	07
RRC	0F
RAL	17
RAR	1F

Arith & Logical Immediate

ADI byte	C6
ACI byte	CE
SUI byte	D6
SBI byte	DE
ANI byte	E6
XRI byte	EE
ORI byte	F6
CPI byte	FE

BRANCH CONTROL GROUP

Jump

JMP adr	C3
JNZ adr	C2
JZ adr	CA
JNC adr	D2
JC adr	DA
JPO adr	E2
JPE adr	EA
JP adr	F2
JM adr	FA
PCHL	E9

Call

CALL adr	CD
CNZ adr	C4
CZ adr	CC
CNC adr	D4
CC adr	DC
CPO adr	E4
CPE adr	EC
CP adr	F4
CM adr	FC

Return

RET	C9
RNZ	C0
RZ	C8
RNC	D0
RC	D8
RPO	E0
RPE	E8
RP	F0
RM	F8

Restart

RST 0	C7
RST 1	CF
RST 2	D7
RST 3	DF
RST 4	E7
RST 5	EF
RST 6	F7
RST 7	FF

I/O AND MACHINE CONTROL

Stack Ops

PUSH	B	C5
	D	D5
	H	E5
	PSW	F5
POP	B	C1
	D	D1
	H	E1
	PSW*	F1
XTHL		E3
SPHL		F9

Input/Output

OUT byte	D3
IN byte	DB

Control

DI	F3
EI	FB
NOP	00
HLT	76

New Instructions (8085 Only)

RIM	20
SIM	30

ASSEMBLER REFERENCE (Cont.)

Pseudo Instruction

General:

ORG
END
EQU
SET
DS
DB
DW

Macro:

MACRO
ENDM
LOCAL
REPT
IRP
IRPC
EXITM

Relocation:

ASEG	NAME
DSEG	STKLN
CSEG	STACK
PUBLIC	MEMORY
EXTRN	

Conditional Assembly:

IF
ELSE
ENDIF

RESTART TABLE

Name	Code	Restart Address
RST 0	C7	0000_{16}
RST 1	CF	0008_{16}
RST 2	D7	0010_{16}
RST 3	DF	0018_{16}
RST 4	E7	0020_{16}
TRAP	Hardware* Function	0024_{16}
RST 5	EF	0028_{16}
RST 5.5	Hardware* Function	$002C_{16}$
RST 6	F7	0030_{16}
RST 6.5	Hardware* Function	0034_{16}
RST 7	FF	0038_{16}
RST 7.5	Hardware* Function	$003C_{16}$

*NOTE: The hardware functions refer to the on-chip interrupt feature of the 8085 only.

Translating Assembly Language Program to Machine Language

- Manual/Hand Assembling
  - Write an assembly language program using mnemonics
  - From the instruction set find the binary code for that mnemonic
  - Write the address for that instruction
  - Write the binary code for the mnemonics
  - Write the operands for the instruction
  - Find the size of that instruction and calculate the address for next instruction
  - Repeat the process for all the mnemonics.

Translating Assembly Language Program to Machine Language (Contd..)

- Automatic Assembling
  - Assembler software is used to convert the assembly language program to machine language
  - Write a program in a text editor
  - The assembly language program file is provided as input to the assembler
  - The assembler generates the machine language file
  - If any library is used linking is done to generate the executable file

An Example of Assembly Language Program

- A simple assembly language program

```
MVI A, 34
MVI B, 79
ADD B
HLT
```

- Program to display the output

```
MVI A, 80
OUT 43
MVI A, 34
MVI B, 79
ADD B
OUT 40
HLT
```

Manual Assembling of Example Programs

Assembling Language Program		Machine Language Program		
START:	MVI A, 34	8000	3E	34
	MVI B, 79	8002	06	79
	ADD B	8004	80	
	RST 5	8005	EF	

Manual Assembling of Example Programs (Contd…)

Assembling Language Program		Machine Language Program		
START:	MVI A, 80	8000	3E	80
	OUT 43	8002	D3	43
	MVI A, 34	8004	3E	34
	MVI B, 79	8006	06	79
	ADD B	8008	80	
	OUT 40	8009	D3	40
	RST 5	800B	EF	

Program Entry Process in Trainer Kit/Simulators

- After writing and manually assembling the program do the following steps to enter the program into the memory of the trainer kit
  - Press key EXAM MEM/SET MEM
  - Enter program start address first
  - Press key NEXT/INR
  - Enter instruction/data
  - Press key NEXT/INR
  - Enter instruction/data
  - Press key NEXT/INR
  - After finishing all the instruction/data press key RESET

Check the Entered Program in Trainer Kit/Simulators

- After entering the program in trainer kit do the following steps to check whether the entry is done correctly or not
  - Press key EXAM MEM/SET MEM
  - Enter program start address
  - Press key NEXT/INR
  - Check whether instruction/data in that address is correct or not
  - Press key NEXT/INR
  - Check whether instruction/data in that address is correct or not
  - Press key NEXT/INR
  - After checking all the locations press key RESET

Executing the Entered Program in Trainer Kit

- After entering and checking the entered program in trainer kit program can be executed
- Program execution can be done in two modes a) Full speed mode b) Single step mode
- To execute the program in full speed do the following steps
  - Press key GO
  - Enter the beginning address of the program
  - Press key EXEC

Executing the Entered Program in Trainer Kit (Contd…)

- To execute the program in single step do the following steps
  - Press key SINGLE STEP
  - Enter Address in Address field
  - Press key EXEC
  - Press key EXAM REG
  - Press respective key A or B to see the value of the register
  - Press key EXEC
  - Press key SINGLE STEP
  - Press key NEXT
  - Press key EXEC
  - Press key EXAM REG
  - Press respective key A or B or EXAM MEM to see the value of the register or memory
  - Press key EXEC
  - Press key SINGLE STEP to resume single step operation
  - After completing the execution press key RESET

Lab Sessions

- There are 10 lab sessions in this course.
- Initial report of each lab is essential while attending the lab class.
- Initial report should contain the programs (mnemonics with op-codes) of all given programs as well as problems and assignments in the lab sheet.
- After the completion of a lab the report of lab must be submitted in the next lab

Lab Exprements

- Lab 1 - Familiarization with 8085 kit (No need for initial and report)
- Lab 2 - Data transfer instructions
- Lab 3 - Arithmetic operations
- Lab 4 - Logical operations
- Lab 5 - Branching and Stack
- Lab 6 - Miscellaneous problems in 8085
- Lab 7 - Familiarization with DOS DEBUG /Assembly Language (No initial)
- Lab 8 - Assembly language programming INT 21H Service
- Lab 9 - Assembly language programming INT 10H Service
- Lab 10 - Assembly language programming miscellaneous Problems
- Lab 11 - Lab Exam

Practical Marks Division

- The marks division for the practical is as follows:

Regularity/Performance/Initial report	10
Final report of each module	10
Viva /Objective test	15
Final lab test	15
Total	50

Executing the Entered Program in Simulators

- Different simulators have different ways provided for program execution.
- For full speed execution simulators may provide different sequences.
- Some simulators provide an area where we enter the starting address of the program and press run on full speed or step by step mode
- Some simulators provide the exact sequence as in the physical single board computer (or trainer kit)
- For single step (step by step) execution simulators may provide menu item or a button to set the execution in single step mode.
  - We can provide the starting address of the program then press the step by step key (or menu item) or some key sequence in keyboard of a computer one at time.

More Exercise

- Enter the following program and run in single step and full speed mode. Check the output in both the cases.

```
8000    MVI A, 80
8002    OUT 43
8004    MVI A, 01
8006    OUT 40
8008    RLC
8009    JMP 8006 (06 80)
800C    RST 5
```